ARTICLES
https://doi.org/10.1038/s41928-022-00774-2
nature electronics

Check for updates

Massively parallel probabilistic computing with sparse Ising machines

Navid Anjum Aadit[1]✉, Andrea Grimaldi[2], Mario Carpentieri[3], Luke Theogarajan[1], John M. Martinis[4,5,6], Giovanni Finocchio[2]✉ and Kerem Y. Camsari[1]✉

Solving computationally hard problems using conventional computing architectures is often slow and energetically inefficient. Quantum computing may help with these challenges, but it is still in the early stages of development. A quantum-inspired alternative is to build domain-specific architectures with classical hardware. Here we report a sparse Ising machine that achieves massive parallelism where the flips per second—the key figure of merit—scales linearly with the number of probabilistic bits. Our sparse Ising machine architecture, prototyped on a field-programmable gate array, is up to six orders of magnitude faster than standard Gibbs sampling on a central processing unit, and offers 5–18 times improvements in sampling speed compared with approaches based on tensor processing units and graphics processing units. Our sparse Ising machine can reliably factor semi-primes up to 32 bits and it outperforms competition-winning Boolean satisfiability solvers in approximate optimization. Moreover, our architecture can find the correct ground state, even when inexact sampling is made with faster clocks. Our problem encoding and sparsification techniques could be applied to other classical and quantum Ising machines, and our architecture could potentially be scaled to 1,000,000 or more p-bits using analogue silicon or nanodevice technologies.

Markov chain Monte Carlo (MCMC) algorithms have had a considerable impact on computing[1], and such methods are among the most powerful randomized algorithms with applications in artificial intelligence[2]. MCMC methods such as Metropolis and Gibbs sampling have, in particular, been widely applied in training generative neural networks[3], probabilistic inference in belief networks[4], calculating physical observables in classical and quantum systems[5], and solving computationally hard combinatorial optimization problems[6].

Designing domain-specific hardware to accelerate these computationally hard artificial intelligence problems has received increasing attention as the computational gains achieved via Moore's law have begun to slow, and a number of different approaches to build special-purpose hardware to solve computationally hard problems have been developed. A class of such solvers (also known as Ising machines) specifically solve quadratic energy models or the Ising model, typically mapped to problems in non-deterministic polynomial time[7–20] as

$$E = -\left(\sum_{i<j} J_{ij} m_i m_j + \sum h_i m_i\right), \tag{1}$$

where quadratic terms ($m_i m_j$) in energy translate to a linear interconnection matrix (J_{ij}) and linear terms (m_i) to a bias vector (h_i), that can be expressed as a graph.

$$I_i = \sum J_{ij} m_j + h_i \tag{2}$$

Choosing the activation function of individual probabilistic bits (p-bits) as

$$m_i = \mathrm{sgn}\left[\tanh(\beta I_i) - \mathrm{rand}_{\mathrm{U}}(-1, 1)\right] \tag{3}$$

ensures that the system states $\{m\}$ are visited according to their corresponding Boltzmann probability:

$$p_i \propto \exp\left[-\beta E(\{m\})\right], \tag{4}$$

where β is introduced as an inverse temperature and can be used to enhance or suppress probabilities corresponding to the energy minima. The dynamical evolution of equations (2) and (3) enables probabilistic sampling and inference, learning weights of a stochastic neural network or performing search or optimization in the exponential state space of the model. Such a machine evolving to the Boltzmann distribution defined by equation (4) is called a Boltzmann machine[3].

Dedicated Ising machines have focused on optimization problems, with the exception of D-Wave's quantum annealers, which have been applied to problems beyond combinatorial optimization[21]. Typically, in Gibbs sampling, equations (2) and (3) are iteratively updated to dynamically evolve the Markov chain such that the network eventually reaches the Boltzmann distribution defined by equation (4). A practical difficulty lies in the serial nature of this evolution. Connected nodes need to be updated one after the other since parallel updating leads to repeated oscillations in the network state, preventing the network from converging to the Boltzmann distribution. The need for sequential updating inherently serializes the network evolution, signified by the nested for loops in the standard descriptions of Gibbs sampling[22].

In this Article, we report a sparse Ising machine (sIM) for solving combinatorial optimization and probabilistic sampling problems, which uses a combination of algorithmic and architectural methods to overcome the serial nature of Gibbs sampling. Our implementation is prototyped on a field-programmable gate array (FPGA), but the architecture is general and could be

[1]Department of Electrical and Computer Engineering, University of California, Santa Barbara, CA, USA. [2]Department of Mathematical and Computer Sciences, Physical Sciences and Earth Sciences, University of Messina, Messina, Italy. [3]Department of Electrical and Information Engineering, Politecnico di Bari, Bari, Italy. [4]Department of Physics, University of California, Santa Barbara, CA, USA. [5]Quantala, Santa Barbara, CA, USA. [6]Zyphra Technologies, San Francisco, CA, USA. ✉e-mail: maadit@ece.ucsb.edu; giovanni.finocchio@unime.it; camsari@ece.ucsb.edu

implemented in digital complementary metal–oxide–semiconductor (CMOS) or energy-efficient nanodevices such as magnetic tunnel junctions (MTJ)[23].

The sIM achieves near-ideal parallelism as long as the interconnections are sparse. To facilitate hardware-friendly problem encoding, we introduce graph sparsification techniques using auxiliary nodes (p-bits). Our architecture exploits the sparsity of the graph through graph colouring and phase-shifted clock to perform exact Gibbs sampling. We find that even when the sampling is made inexact through faster clocks, the network often finds the exact ground states in hard optimization problems.

In the sampling throughput, our architecture achieves highly competitive performance against implementations based on an optimized tensor processing unit (TPU) and graphics processing unit (GPU). In hard optimization problems such as integer factorization, our architecture reliably factors semi-primes up to 32 bits, far larger than what has been reported for D-Wave's quantum annealers or similar probabilistic solvers[13,16,18,24–26]. Finally, our architecture outperforms competition-winning Boolean satisfiability (SAT) solvers in approximate optimization.

Parallelized architecture

Combinatorial optimization in dedicated Ising machines are typically solved by heuristic interconnection matrices[27]. In this Article, we use the invertible logic formulation[28,29] where a universal set of invertible AND, OR and NOT gates can be composed to systematically formulate cost functions for any given optimization problem (Supplementary Information provides details regarding invertible logic and reconfigurability). Hard combinatorial optimization problems such as integer factorization and SAT can be solved in hardware using invertible multipliers or by invertible Boolean circuits corresponding to a given satisfiability instance (Fig. 1a,b).

Given an interconnection matrix obtained by invertible logic, we colour the resultant graph using the heuristic graph-colouring algorithm DSATUR (ref. [30]). The colouring is used to exploit parallel updating of unconnected (conditionally independent) p-bits (Fig. 1c). Colouring a graph with the minimum possible colours is non-deterministic polynomial-time hard; however, this is unimportant for our purposes since we use DSATUR as a greedy algorithm that may or may not find the optimum colouring.

We find that when the overall graph density is low (for example, $\lesssim$1%), irregular graphs containing nodes with hundreds of neighbours can be coloured by a few colours. For example, for the 8-bit factorizer graph, only five colours are used; as a result, we need five parallel and equally phase-shifted clocks in the sIM (Fig. 1d). The equal phase shift is required to avoid any concurrent edges of the clocks. In our architecture (Fig. 1d), different colour blocks receive different clocks to their random number generators (RNG), ensuring no neighbouring p-bits flip at the same time for exact Gibbs sampling. This is a constraint that we relax later to perform inexact Gibbs sampling (Fig. 5), reminiscent of the HOGWILD!–Gibbs algorithm[31].

The idea of block updating in Gibbs/Metropolis sampling is commonly used for regular graphs. When the graph is bipartite (as in restricted Boltzmann machines or chessboard lattices), trivial colourings (with two colours (black and white) or four colours in King's graphs) are possible, and this is often exploited in updating each colour block in parallel[4,32–35]. Compared with prior work on block updating, our contributions are twofold: first, we extend the block-updating scheme such that it applies to regular and irregular graphs with the only requirement that the graphs are sparse enough to be coloured by a few colours (typically $\lesssim$4–8). This generalization is notable, considering most practical instances of combinatorial optimization problems may have irregular graph representations. Second, we provide exact sparsification methods that can enable parallelism in dense problem graphs. We believe that both methods can be useful for other Ising machines and different problems.

We also note that parallelization techniques in multiple FPGAs have recently been explored for other types of Ising machine[20]; however, these are based on different algorithms unrelated to our computational model based on equations (2)–(4).

In the case of exact Gibbs sampling, our architecture ensures the entire network is updated in parallel in one clock period (of any colour) and ensures effectively sequential updates. As a measure of sparsity, we use graph density ρ. For an undirected graph, ρ is defined as

$$\rho = \frac{2|E|}{|V|^2 - |V|}, \tag{5}$$

where $|V|$ is the number of nodes (vertices), corresponding to the number of p-bits in the sIM, and $|E|$ is the number of edges, corresponding to the interconnections in the $[J]$ matrix. We find that when the p-circuits are composed using invertible logic, both factorization and satisfiability graphs are sparse (Supplementary Fig. 5). For example, consider a 32-bit factorizer (Fig. 1e) having 784 p-bits with $\rho = 1.03\%$, requiring only five distinct colours. In this example, five parallel and equally phase-shifted clocks in the sIM are adequate to implement this p-circuit for massively parallel computation.

Graph sparsification

There is a limitation on the clock speed depending on the maximum number of neighbours for each p-bit. The neighbour distribution of the 32-bit factorizer reveals that it has 32 p-bits with 32 neighbours (Fig. 1e). To ensure every single colour block is updated with the latest state of each neighbour, the multiply-accumulate (MAC) unit implementing equation (2) needs to finish computation before the next colour block is updated. As such, the addition for the weights needs to be completed within the nth of a clock period (n is the number of colours). In the example shown in Fig. 1e, this requires large adders to add 32 s-bit numbers within this short period, where s is the bit precision of the weights. Note that for p-bits, multiplication does not introduce delays, since the weights are either selected or ignored as the p-bit states are either 0 or 1.

To overcome the limitation introduced by adder delays, we developed exact sparsification techniques to remove the nodes with a large number of neighbours, keeping the structure of the optimization problem intact. The main idea is to split a given p-bit representation between two p-bits coupled by a ferromagnetic ($J > 0$) interaction, which we call a COPY gate[36]. The ferromagnetic interaction ensures that at the end of an annealing schedule (high β), the ground states of the split and fused models are identical (Supplementary Information provides a formal proof).

Figure 1f shows the sparsified graph of the 32-bit factorizer p-circuit with 2,128 p-bits and a graph density of 0.2%. It requires five colours as before; however, the neighbour distribution reveals the maximum number of neighbours k is limited to 5. Sparsification always introduces extra p-bits; however, in scaled implementations, individual p-bits are almost always cheaper than complicated interconnects.

Similarly, the original graph of a 100-variable 3SAT 'uf-100-01.cnf' instance can be coloured using seven colours (Fig. 1g) in a sparse graph with 531 p-bits and $\rho = 1.55\%$. A sparsified version of this graph is shown in Fig. 1h with 1,935 p-bits and a graph density of 0.2%. As before, the sparsification ensures the graph has a maximum of $k = 4$ neighbours for each p-bit and hence avoids large adder delays, requiring four colours (clocks).

Even though we show specific cases of sparsification (with $k = 4$ neighbours for satisfiability and $k = 5$ for factorization), we have systematically analysed the effect of sparsification (as a function of k) on the system size and performance (Supplementary Information). Limiting the number of neighbours per p-bit to a fixed value

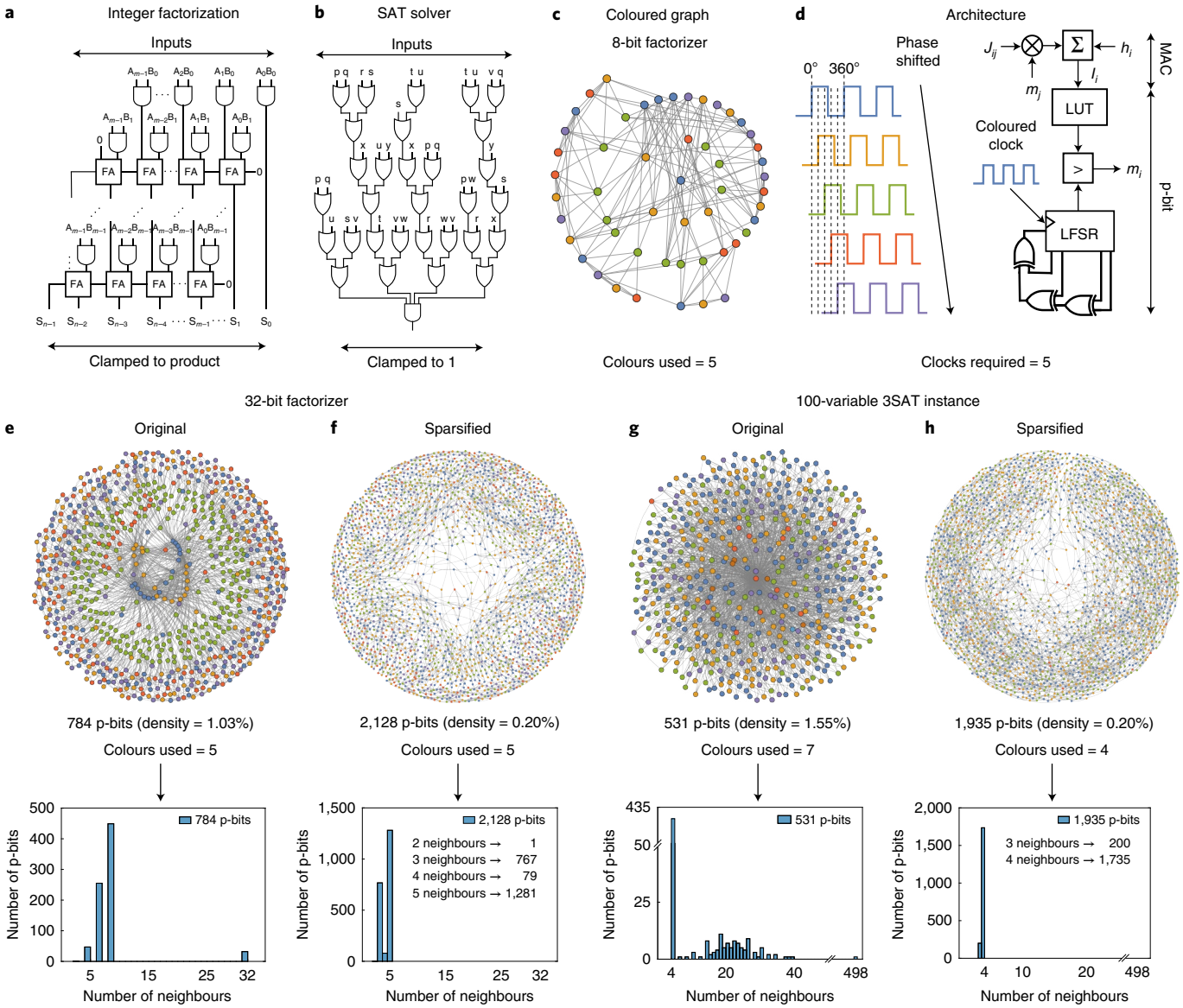

Fig. 1 | Combinatorial optimization with invertible logic. a, Digital multiplier circuit composed of AND gates and full adders (FAs) that can be invertibly run to solve *n*-bit integer factorization problem. **b**, Boolean circuit composed of OR and AND gates that can be invertibly run to solve satisfiability problems. **c**, Example 8-bit factorizer graph coloured with five colours. **d**, Example architecture with five parallel and equally phase-shifted clocks, implementing the multiply-accumulate (MAC) (equation (2)) and p-bit (equation (3)) equations. The activation function is implemented using a lookup table (LUT) and the random number is generated using a linear-feedback shift register (LFSR). **e**, Original graph of a 32-bit factorizer developed using invertible logic requires five colours, but it contains nodes with up to 32 neighbours, limiting hardware implementations. **f**, Exact sparsification techniques are used to reduce the nodes with a large number of neighbours at the expense of additional bits. **g**, Original version of a 100-variable 3SAT instance developed using invertible logic. **h**, Sparsified version with nodes having a maximum of only four neighbours. Graph density ρ is defined by equation (5).

($k=4,8\ldots$) ensures that the adder delays do not grow with system size. In other words, no matter how large the global system becomes, only the local neighbourhood of a p-bit (with k-neighbours) needs to communicate faster than the p-bit clocks, ensuring scalability.

We implemented the sIM architecture on a Xilinx Virtex UltraScale+ FPGA VCU118 evaluation board for our experiments. Supplementary Information provides details of the FPGA architecture and its design choices.

Comparisons among CPU, GPU and TPU

To test the parallelism achieved by our approach, we compared our sIM with standard Gibbs sampling[22] implemented on a CPU. Our purpose is to stress the asymptotic scaling differences between the standard approach and ours, rather than a pure performance comparison that we perform later against optimized GPU and TPU implementations.

As a model problem, we define approximate factorization as reaching 99% of the absolute ground energy from 14-bit to 50-bit semi-primes. We do not attempt exact factorization since the absolute ground state can be difficult to reach in simulated annealing and the CPU practically never reaches there. Although finding approximate factors is not useful for factoring, many other optimization problems benefit from high-quality and approximate solutions. As such, we treat approximate factorization as a computationally hard benchmark.

We use the same annealing schedule and the same sparsified graphs for the comparison between an unoptimized serial MATLAB

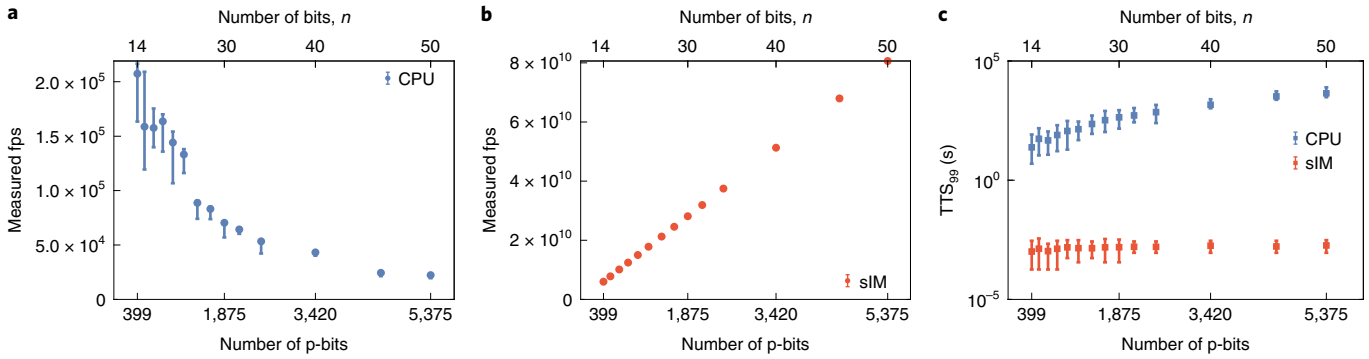

Fig. 2 | Performance scaling between parallel (sIM) and serial (CPU) implementations. a, CPU fps as a function of graph size; larger graph sizes drastically limit the fps. **b**, Measured sIM fps as a function of graph size, showing ideal parallelism scaling linearly with the system size in contrast to the serial CPU. **c**, Time-to-solution comparison for approximate factorization between the CPU and sIM as a function of graph size. TTS_{99} is defined as the time required to reach 99% of the ground state. TTS_{99} requires up to 4,408.93 s for the 50-bit factorization in the CPU. For the sIM, it shows an almost flatline behaviour with up to ~10^6 times performance improvement.

Table 1 | Sampling throughput

Sampling throughput			Integer factorization	
Platform	**Graph**	**Flips per nanosecond**	**Platform**	**Integers factored (up to)**
			D-Wave 2000Q (ref. [24])	143 (8 bit)
NVIDIA Tesla C1060 GPU[37,38]	Chessboard	7.98	CMOS invertible logic[18]	598 (10 bit)
NVIDIA Tesla V100 GPU[34]	Chessboard	11.37	Stochastic MTJ[13]	945 (10 bit)
Google TPU[34]	Chessboard	12.88	FPGA RBM[16]	43,621 (16 bit)
NVIDIA Fermi GPU[33]	Chessboard	29.85	D-Wave 2000Q (ref. [25])	223,357 (18 bit)
			D-Wave 2000Q (ref. [26])	249,919 (18 bit)
FPGA sIM (this work)	Irregular	143.80	FPGA sIM (this work)	4,277,546,633 (32 bit)
Nanodevice sIM (projected)[13,40–42]	Irregular	1,000,000		

Comparison of FPGA-based and nanodevice-based (projected) sIMs with optimized GPU and TPU implementations of MCMC sampling. Unlike GPUs and TPUs, the sIM can support regular chessboard lattices as well as the irregular graphs discussed here. For the sIM, fps is a size-dependent metric (Figs. 2 and 4). In this table, the best fps achieved in the largest system is quoted. Integer factorization: comparison between state-of-the-art hardware factorizers and FPGA-based sIM. Note that all these solvers treat integer factorization as a frustrated spin-glass problem and perform classical or quantum annealing (or ordinary sampling). We show the best reported numbers and single instances; in the case of sIM, random semi-primes up to 32 bits can be reliably factored (as shown in the main text).

(version 2020b) implementation on a CPU and the parallel FPGA implementation of the sIM. For each problem, we have attempted to factor ten different numbers ten times to collect enough statistics.

As a key metric, we focus on the flips per second (fps) as a figure of merit. The fps value corresponds to the correlated fps that can be taken by the system where every flip is made based on the latest state of the network between physically connected nodes and avoiding simultaneous updates. Several hardware implementations for MCMC solvers have reported this metric[33,34,37,38]. The key point of the parallel architecture designed for the sIM here is that its sampling throughput (fps) increases linearly with the number of p-bits.

Figure 2a,b presents a comparison of fps between the inherently serial CPU and the sIM. The CPU calculation is done in MATLAB by an iterative solution of equations (2) and (3) using standard Gibbs sampling[22]. Even though optimization techniques including graph colouring by multiple threads could improve our MATLAB implementation, our purpose is to stress the massive parallelism achieved by the sparse architecture that we have developed over a standard implementation of Gibbs sampling.

MATLAB runs on the Knot Cluster at the Center for Scientific Computing server, University of California, Santa Barbara, featuring an Intel Xeon Processor E5630 running at up to 2.8 GHz. For sIM, we used five equally phase-shifted 15 MHz clocks and assigned those clocks to the p-bits in the coloured graph. In the sIM, each p-bit updates in parallel, achieving an effective clock speed of $N \times 15$ MHz, where N is the number of p-bits. We chose 15 MHz to satisfy the timing; however, we envision that more powerful FPGAs or custom hardware can reach much higher speeds.

We directly measured the fps of the sIM by means of specially designed reference counters in the FPGA (Methods provides the measurement details). The maximum fps achieved by the CPU is limited to around 10^5 (Fig. 2a). In the CPU, fps quickly starts to decline with an increasing number of p-bits mainly due to the nested for loops in standard Gibbs sampling[22].

On the contrary, the sIM collects up to 80–100 billion samples per second at the highest problem sizes. Most importantly, the fps increases linearly with the increasing number of p-bits (Fig. 2b) due to its massively parallel architecture. Starting at an fps of 5.99×10^9 for 14-bit factorization, it achieves a maximum fps of 8.06×10^{10} for the 50-bit factorization problem.

An important question is whether all these samples are useful or not. To test this, we define the time to solution (TTS_{99}) as the time it takes to reach 99% of the absolute ground energy. In the case of factorization, the exact solution is planted and we have access to the exact ground-state energy. Figure 2c shows a steep rise in TTS_{99} from 14- to 50-bit factorization for the CPU. The fastest case requires TTS_{99} of 23.84 s for 14-bit factorization, whereas the slowest requires 4,408.93 s for 50-bit factorization. In contrast, the sIM shows a roughly constant mean value for all the problems, requiring 1.02 ms for 14-bit factorization and 1.84 ms for 50-bit

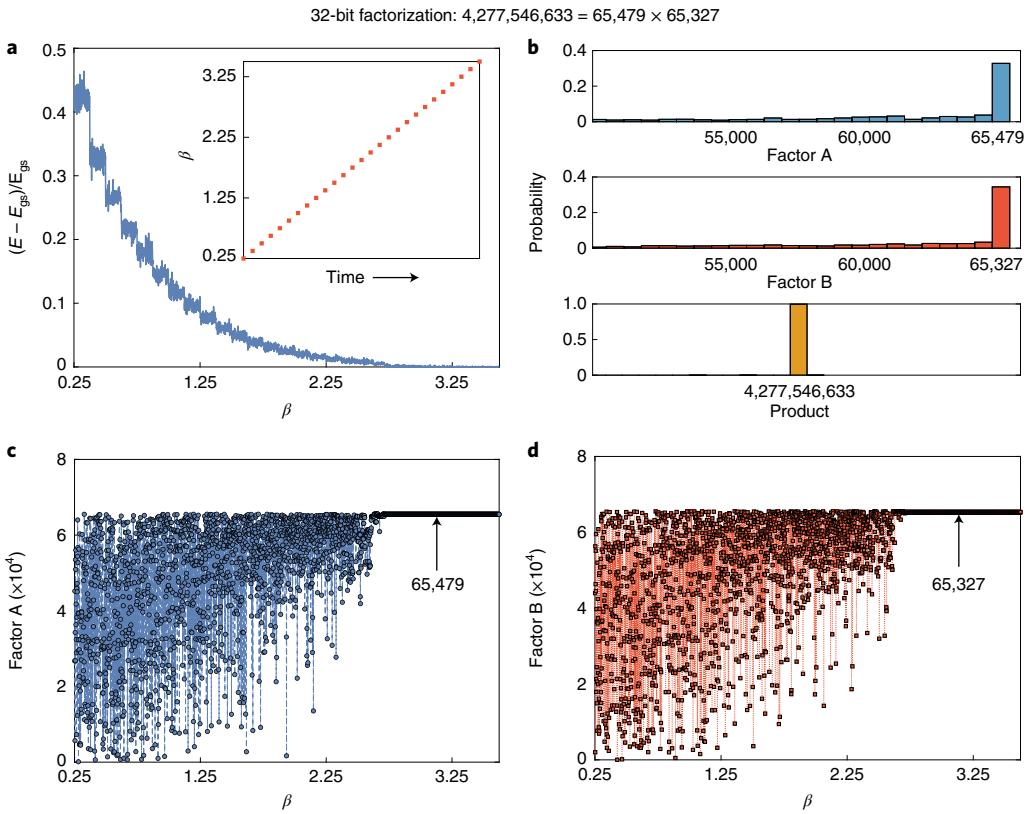

Fig. 3 | Exact factorization of a 32-bit number (P = 4,277,546,633) in the sIM. a, Normalized energy as a function of inverse temperature (β from equation (4)) where Egs refers to the energy of the ground state. The inset shows the linear annealing schedule. **b**, Histograms of product P and factors A and B over the entire annealing schedule. **c**,**d**, Factors A (**c**) and B (**d**) at different β values, showing that they converge to the right ground state at the highest β.

factorization (Methods provides the TTS measurement details). The 50-bit factorization process has an ~2.4×10^6 times improvement over the CPU. The reason for the constant time to solution for the sIM despite the increasing difficulty of the problem is because increasing the problem size also increases the fps. We conclude that the measured fps (Fig. 2b) directly translates to an improvement in TTS.

Table 1 summarizes the performance benchmarking of sIM against state-of-the-art GPUs and TPUs. An important distinction between these comparisons is that virtually all the optimized implementations of GPUs and TPUs make use of a regular two-dimensional lattice (chessboard) where partitioning the graph into two colour blocks becomes the key piece that enables parallelism. By contrast, in our examples, we show that for realistic instances of combinatorial optimization problems using invertible Boolean logic, the resulting graphs are sparse but not necessarily regular or bipartite.

The fps performance of the sIM grows linearly with the number of p-bits in a network; hence, the fps becomes a size-dependent metric. We observe that the sIM reaches up to 143.8 flips ns^{-1} at one of the largest graph nodes (4,793 p-bits). This is an 18.03 times performance gain over a single NVIDIA Tesla C1060 GPU with multi-spin coding[37,38]. It also outperforms the Google Cloud TPU implementation of the two-dimensional Ising model by a factor of 11.17 and its reference NVIDIA Tesla V100 GPU by a factor of 12.65 (ref. [34]). Furthermore, the sIM provides 4.82 times more flips per nanosecond than the NVIDIA Fermi GPU coded for simulating the three-dimensional Edwards–Anderson model with parallel tempering[33].

Beyond the FPGA-based sIM implementation considered here, in this paper, we make performance projections for massively parallel asynchronous sIMs using nanodevices (Supplementary Fig. 1). MTJs have recently attracted attention as building blocks for probabilistic computation because of their extreme scalability. The magnetic memory industry has integrated up to billions of single MTJs to replace various parts of the memory hierarchy[39]. Through minimal modifications, such MTJs can be made stochastic[13], providing expensive random-number generation with a negligible hardware cost. Stochastic MTJs have been demonstrated to provide fast fluctuations ($\tau = 1$ ns $flip^{-1}$)[40,41] and at least a million MTJs ($N = 10^6$) can be integrated in massively parallel architectures[13,42] similar to what we consider in this paper. Following the linear scaling law (Fig. 2b), such sIMs can provide N/τ flips per second, reaching 1 million flips per nanosecond (Table 1), provided that the hardware connectivity is sparse enough to enable the ideal parallelism demonstrated in this Article.

Exact factorization

Beyond approximate factorization, we also performed exact factorization, to benchmark our approach against other probabilistic solvers and D-Wave. We found that the sIM can reliably factor random semi-primes up to 32 bits (Supplementary Fig. 7). To the best of our knowledge, this result is the largest among all the other approaches of solving factorization as an optimization problem[13,16,18,24–26]. Table 1 presents a comparison of integer factorization across state-of-the-art hardware platforms. The commonality in all these solvers is that they express factorization as a frustrated spin-glass problem in which the ground state is searched using classical or quantum annealing.

From an algorithmic perspective, factorization of 32-bit semi-primes is not difficult since even with trial division, this is relatively easy. From a statistical physics perspective, however, finding

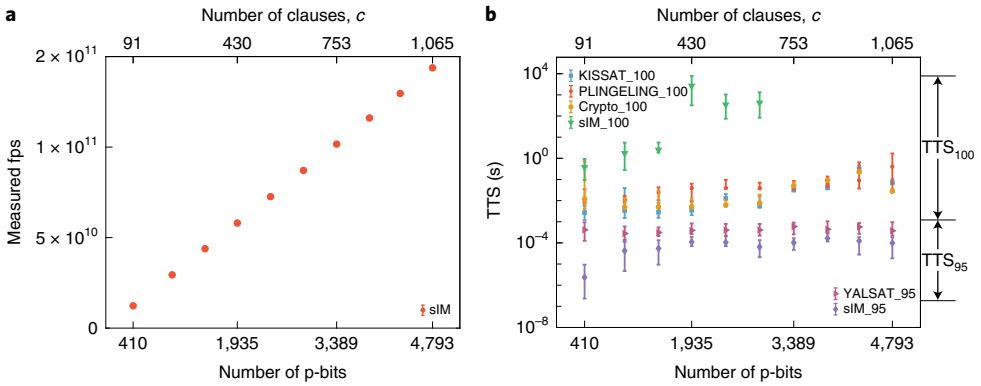

Fig. 4 | Performance of sIM versus competition-winning SAT solvers. a, The fps values of the sIM increases linearly with the graph size, showing ideal parallelism. The sIM achieves a record fps value of 1.44×10^{11} for the largest problem with 4,793 p-bits and 1,065 clauses. **b**, Run times to solve the UBC SATLIB (ref. [45]) 3SAT instances. Winning solvers from the SAT 2020 competition, namely KISSAT, PLINGELING and CryptoMiniSat[46,47] find 100% solution (all the clauses are satisfied). SAT competition 2017 random track winner YALSAT[49] is set to find the 95% solution (95% of the clauses are satisfied). In exact SAT solving, the sIM is slower than all the professional SAT solvers. However, in approximate SAT solving to satisfy 95% of the clauses, the sIM outperforms all these solvers (including local search-based SAT solvers, such as YALSAT), delivering the fastest solution.

the ground state of a frustrated spin glass in 2^N dimensional space ($N>2{,}000$ p-bits) is striking. Contrasting the time to solution shown for approximate factorization (Fig. 2c) to that of exact factorization (Supplementary Fig. 7) as a function of problem size, we observe a drastic difference in algorithmic scaling, indicative of a 'golf course'-like energy landscape where the ground state is well hidden from the approximate states that are easy to reach.

It is worth stressing that we used a simple, standard simulated annealing algorithm without any fine tuning or optimization. Combining our architecture with powerful algorithmic ideas[43] could improve these results further.

Figure 3 presents the exact factorization of a 32-bit number, namely, P = 4,277,546,633. The linear annealing schedule and normalized energy of the system are presented in Fig. 3a. The absolute ground state is reached at the coolest temperature (highest β). The histograms of product P and factors A and B over the entire annealing schedule are presented in Fig. 3b where the exact factors A = 65,479 and B = 65,327 are reliably visited. Fig. 3c,d reconfirms that the factors are consistently found at the highest β value without any fluctuations.

Although we do not show the statistics in Fig. 3, Supplementary Fig. 7 reports the time to find the exact factors (TTS_{100}) from 14-bit to 32-bit semi-primes. For any of these problems, the CPU fails to find the exact factors even over a very long time and therefore is excluded from the TTS_{100} report and we report the sIM times. As before, we attempt to factor ten different numbers ten times for each problem.

Unlike approximate factorization where we defined TTS_{99} as the average time the sIM takes before reaching 99% of the absolute ground state, we find that for exact factoring, there is exponential dependence of time with respect to the problem size. Nevertheless, improving SAT solving with massively parallel hardware could still be useful in accelerating critical subroutines of the best factoring algorithms[44].

Boolean satisfiability with invertible logic

Next, we compared our sIM with competition-winning SAT solvers for the problem of Boolean satisfiability (3SAT). As discussed earlier, we first report the fps achieved by the sIM for different 3SAT instances defined by the number of their clauses (Fig. 4a). The sIM runs with four parallel and equally phase-shifted clocks operating at 30 MHz since in this case, the sparsified graphs require only four colours. For the smallest 'uf20-01.cnf' instance with 20 variables and 91 clauses, the sIM achieves an fps of 1.23×10^{10} with 410 p-bits. For the largest 'uf250-01.cnf' instance with 250 variables and 1,065 clauses, the sIM achieves a record fps value of 1.44×10^{11} with 4,793 p-bit, a 5–18 times speed improvement over optimized TPUs and GPUs (Table 1).

Figure 4b shows the run times to solve the UBC SATLIB (ref. [45]) 3SAT instances using different professional SAT solvers and the sIM. We solve each instance 100 times. We compare our results with winning solvers from the SAT 2020 competition, namely, KISSAT, PLINGELING and CryptoMiniSat[46,47]. All these solvers are executed on the same Linux machine having a flagship Intel Core i9-10900 Processor running at up to 5.20 GHz. Time to solve all the clauses (100% solution) is reported as $KISSAT_{100}$, $PLINGELING_{100}$ and $Crypto_{100}$, respectively. We have used linear simulated annealing in the sIM to report the time to solve all the clauses, labelled as sIM_{100} (Fig. 4b).

As typical of simulated annealing, reaching the absolute ground state is difficult for the sIM. We report sIM_{100}, namely, the time it takes for sIM to satisfy all the clauses in a given 3SAT instance and find that despite the enormous number of flips per second taken by the massively parallel processor, we did not find the ground state beyond 2,903 p-bits (Fig. 4b; sIM_{100}).

In many practical instances, however, we may not be interested in finding the absolute ground state of an optimization problem, and reaching approximate but practically useful solutions as quickly as possible may be far more important. In this setting, we find that the sIM beats all the mentioned SAT solvers (Fig. 4b; sIM_{95}). Because solvers based on conflict-driven clause learning such as KISSAT, PLINGELING and CryptoMiniSat are programmed to find the exact solution, we also test the sIM against another solver called YALSAT (2017 SAT competition random track winner), which reports the current best solution. We program the solver to stop when it reaches 95% of the solution (denoted as $YALSAT_{95}$). sIM is also set to solve 95% of all the clauses and the time to solution is noted as sIM_{95}. We find that in approximate SAT solving, the sIM provides the fastest approximation, outperforming all the SAT solvers by a factor of 4 to 700. We find that unlike the case of the absolute ground state, there is no failure even for the largest instance ('uf250-01.cnf') that we can fit into our FPGA encoded with 4,793 p-bits. It takes 2.36 µs for the 'uf20-01.cnf' instance and 98.26 µs for the 'uf250-01.cnf' instance to solve 95% of the clauses. Given the linear dependence of fps to the system size, we expect larger improvements in scaled implementations of sIM.

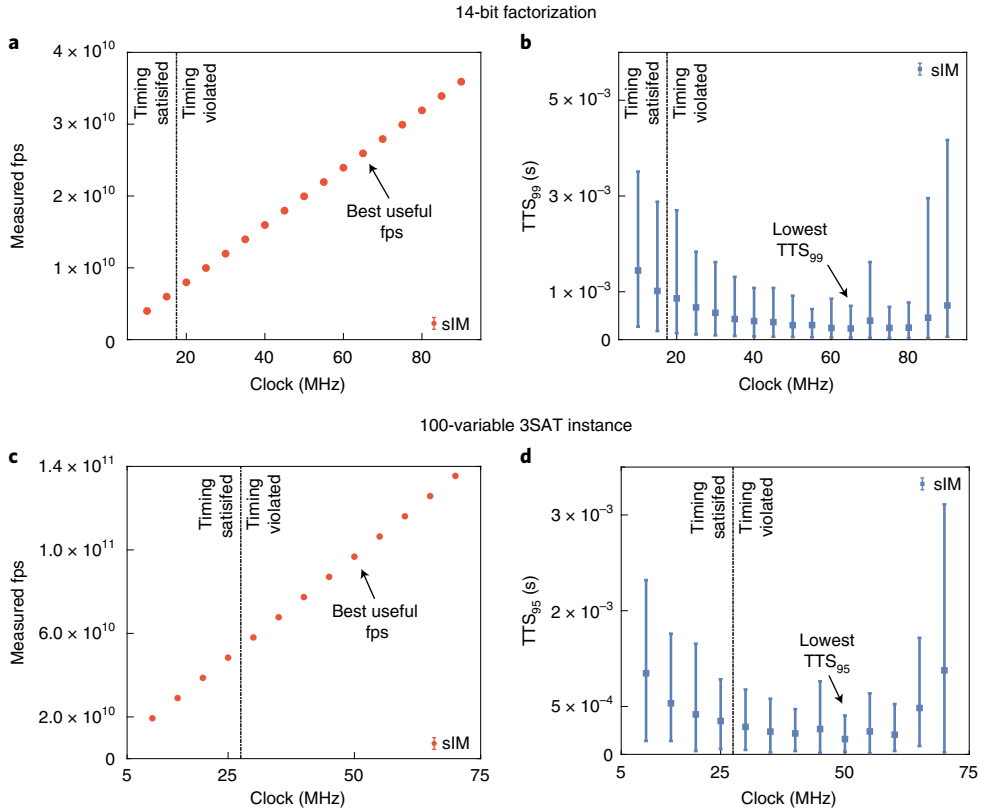

Fig. 5 | Overclocked Gibbs sampling with sIM. a, 14-bit factorization: fps as a function of p-bit clocks. The timing is violated after 15 MHz. Based on **b**, we observe that overclocking effectively improves fps up to 65 MHz. **b**, Time to solution to reach 99% of the ground state, TTS_{99}, measured as a function of clock frequency. **c**, 3SAT 'uf-100-01.cnf' instance: measured fps as a function of p-bit clocks. The timing is violated above 25 MHz. Based on the results of **d**, the highest effective fps is achieved at 50 MHz. **d**, Time to solution to satisfy 95% of the clauses (TTS_{95}) as a function of clock frequencies. The results in both **b** and **d** show qualitatively similar behaviour of improving performance followed by a sharp decline, indicative of a universal behaviour.

Inexact Gibbs sampling

The parallelized sIM architecture introduced here exactly implements equations (2) and (3). This is ensured by making every colour block update with the most up-to-date neighbour information. Since this architecture is inspired by asynchronous and physical p-bit implementations, for example, using stochastic MTJs, a natural question is whether inexact Gibbs sampling where the p-bits do not update with the most up-to-date neighbour information is useful or not. This approach is reminiscent of asynchronous Gibbs sampling (or HOGWILD!–Gibbs) algorithms that have been theoretically analysed[31].

We systematically investigate how increasing individual clock frequencies of colour blocks and eventually performing inexact Gibbs sampling with old statistics at the p-bit level affects the system performance. For two completely different problems (integer factorization and Boolean satisfiability), we observe qualitatively similar results as a function of increasing clock frequency, thereby increasing the number of messages dropped between the neighbours. In both cases (Fig. 5b,d), we observe an initial decrease in the time to solution with increasingly incorrect updates followed by a sharp increase. Increasing the clock frequencies naturally increases the fps of the main network (Fig. 5a,c); however, when the messages are dropped beyond a certain threshold, the improvement in the fps does not help the network converge to the right answers.

To test the generality of overclocking, we analysed inexact Gibbs sampling with systematically introduced errors in a five p-bit full-adder circuit (Supplementary Information). By introducing two different error models, we observe qualitatively similar behaviour, where introducing a small amount of error (related to the number of messages dropped between the neighbours) does not lead to obvious deviations from the exact Boltzmann distribution. This explains why a moderate amount of overclocking is effective: increasing fps without introducing critical errors decreases the time to solution, as observed in the two different problems shown in Fig. 5. Supplementary Information also shows how further overclocking reduces the network to a fully synchronous (parallel) updating state, providing analytical estimates of the limiting behaviour.

One additional reason why overclocking notably improves the performance is due to the slowing down of the network dynamics at lower temperatures (higher β). Despite introducing timing failures in many paths and potentially dropping a substantial number of messages, failing paths are not activated because p-bits do not frequently change their states at low temperatures. The degree of resilience to errors through inexact sampling is an important feature of such probabilistic methods[48], and this feature can be exploited in truly asynchronous nanodevice-based implementations of the sIM[13,40–42].

Conclusions

We have reported a sIM with a massively parallel architecture that can be used to parallelize a broad range of MCMC algorithms for computationally hard problems. In particular, by overcoming the serial nature of MCMC algorithms such as Gibbs sampling, we have developed an architecture that can achieve ideal parallelism where the main metric of the sIM, namely, fps, scales linearly with the number of p-bits in the system. This parallel architecture uses several algorithmic ideas, combining invertible logic to produce sparse graph representations of combinatorial optimization problems such

as Boolean satisfiability and integer factorization. Further sparsification was needed to ensure matrix multiplication and addition could be performed before independent p-bits are updated. The architecture uses approximate graph colouring to parallelize sampling.

We have shown an FPGA-based implementation of this concept achieving three key results. First, comparisons to an ordinary CPU implementation of Gibbs sampling showed that the sIM is able to achieve up to six order of magnitude improvement in fps, which directly translates to advantages in time to solution in the integer factorization problem; in comparison to highly optimized GPU and TPU implementations, the sIM showed up to 5–18 times measured speed improvement in fps, without the use of regular or simple graphs as commonly used for benchmarking purposes in GPUs and CPUs. Second, the sIM was able to factor semi-primes up to 32-bit integers, far larger than the best available results on factoring where an optimization approach is taken. Third, the sIM was able to beat competition-winning SAT solvers in approximate satisfiability, delivering superior performance compared with the best possible classical approach in solving satisfiability problems. We have also shown how overclocking in the spirit of asynchronous Gibbs sampling[31] could lead to performance improvements.

These results were obtained using an FPGA platform, where our problem sizes were limited to the number of p-bits that we could fit in a single device. The use of more powerful FPGAs would immediately extend the size of the problems programmable to the sIM. The parallelism we achieved in the architecture, coupled with algorithmic sparsification techniques we developed, could be further exploited in highly scaled implementations and other Ising machines. In particular, nanodevices (or analogue CMOS-based) p-bits could lead to considerable improvements over our reported results.

Methods

Problem description. For the factorization problem, we generated random semi-prime numbers from 14 to 50 bits using MATLAB. For each instance, ten different numbers were generated. The graphs obtained using invertible logic and sparsification are very sparse (Supplementary Fig. 5a).

For the SAT problem, we solved 3SAT instances (each clause has exactly three variables). The instances were collected as .cnf files from the UBC SATLIB library[45]. Similar to factorization, the 3SAT graphs are very sparse (Supplementary Fig. 5b).

Simulated annealing. In simulated annealing, β is gradually increased over time. According to equation (2), multiplication of β and input weights (J,h) are performed in MATLAB. The updated values of J and h are sent to the FPGA for every β over time to perform simulated annealing.

Data READ/WRITE. MATLAB is used to READ/WRITE data from the FPGA through a USB JTAG interface (Supplementary Fig. 2a). A programmable timer is implemented in the FPGA. Using the timer, a global DISABLE signal is sent to the p-bits before a READ instruction. The timer is preset from the program (MATLAB) and all the p-bits are automatically frozen at the same time when the time is up. Once the p-bits are frozen, the data are READ using the USB JTAG interface and sent to MATLAB for post-processing. When the READ instruction is DONE, the timer is RESET from MATLAB to resume the p-bits, if necessary. Similarly, for the WRITE instruction, a global DISABLE signal is sent using the programmable timer to freeze the p-bits before sending the weights. Similarly, for the READ instruction, the timer is RESET from MATLAB to resume the p-bits after the WRITE instruction is DONE.

Measurement of fps. Each p-bit is designed with a programmable stopwatch counter in the FPGA. A parallelly running global counter is set to count up to a preset value at the positive edge of a known clock. When the global counter is DONE counting, a global DISABLE signal is broadcast to all the other counters. Comparing the p-bit counter outputs (number of flips) with the global counter preset value, the time for the total flips is obtained. With these data, the fps of the sIM is experimentally measured for each p-bit. To measure the fps in the case of the CPU, built-in functions from MATLAB are used to measure the elapsed time and programmatically count the total flips in that time. With these data, the fps is measured in real time. The error bars in all the figures are obtained by taking 100 measurements of fps.

Measurement of TTS. In the FPGA, a minimum time is set using the programmable timer to find the solution to the problem of interest. After that time, a global DISABLE signal is sent to READ the latest TTS. In iterations, the minimum time is incremented, and the p-bits are RESET. This process is repeated until the desired solution is reached. The latest TTS is reported as the TTS of the sIM for that problem. In measuring the TTS, we do not include the READ/WRITE times through the USB JTAG interface. Although we use a slow USB JTAG interface (up to 33 MHz) for the convenience of using MATLAB, much faster READ/WRITE protocols such as PCI Express (up to 8 Gb s^{-1}) would entirely remove this time. To measure the TTS in the case of the CPU, a predefined minimum number of samples is set to find the solution. The number of samples is increased in iterations until the optimum solution is found by the CPU. The time to solution is recorded using the built-in function and the latest one is reported as the TTS of the CPU for that problem. The error bars in all the figures are obtained by taking 100 measurements of TTS.

Setting up the SAT solvers. The online source codes of the SAT solvers are used to build the solvers on a Linux machine. For SAT solvers based on conflict-driven clause learning, the time to find the 100% solution is measured using a simple Python (version 3.8) script. For a local SAT solver (for example, YALSAT), the program is set to report the TTS for the current best solution.

Data availability

The data that support the plots within this paper and other findings of this study are available from the corresponding authors upon reasonable request.

Code availability

The computer code and problem instances used in this study are available from the corresponding authors upon reasonable request.

Received: 14 October 2021; Accepted: 26 April 2022;
Published online: 2 June 2022

References

1. Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H. & Teller, E. Equation of state calculations by fast computing machines. *J. Chem. Phys.* **21**, 1087–1092 (1953).
2. Buluc, A. et al. Randomized algorithms for scientific computing (RASC). Preprint at https://arxiv.org/abs/2104.11079 (2021).
3. Hinton, G. E. A practical guide to training restricted Boltzmann machines. in *Neural Networks: Tricks of the Trade* 599–619 (Springer, 2012).
4. Mansinghka, V. K., Jonas, E. M. & Tenenbaum, J. B. *Stochastic Digital Circuits for Probabilistic Inference*. Report No. MITCSAIL-TR (Massachussets Institute of Technology, 2008).
5. Bouchard-Côté, A., J Vollmer, S. & Doucet, A. The bouncy particle sampler: a nonreversible rejection-free Markov chain Monte Carlo method. *J. Am. Stat. Assoc.* **113**, 855–867 (2018).
6. Kirkpatrick, S., Gelatt, C. D. & Vecchi, M. P. Optimization by simulated annealing. *Science* **220**, 671–680 (1983).
7. McMahon, P. L. et al. A fully programmable 100-spin coherent Ising machine with all-to-all connections. *Science* **354**, 614–617 (2016).
8. Yamaokam, M. et al. 24.3 20k-spin Ising chip for combinational optimization problem with CMOS annealing. In *2015 IEEE International Solid-State Circuits Conference—(ISSCC) Digest of Technical Papers* 1–3 (IEEE, 2015).
9. Goto, H., Tatsumura, K. & Dixon, A. R. Combinatorial optimization by simulating adiabatic bifurcations in nonlinear Hamiltonian systems. *Sci. Adv.* **5**, eaav2372 (2019).
10. Wang, T. & Roychowdhury, J. OIM: oscillator-based Ising machines for solving combinatorial optimisation problems. In *International Conference on Unconventional Computation and Natural Computation* 232–256 (Springer, 2019).
11. Ahmed, I., Chiu, P.-W. & Kim, C. H. A probabilistic self-annealing compute fabric based on 560 hexagonally coupled ring oscillators for solving combinatorial optimization problems. In *2020 IEEE Symposium on VLSI Circuits* 1–2 (IEEE, 2020).
12. Dutta, S. et al. An Ising Hamiltonian solver based on coupled stochastic phase-transition nano-oscillators. *Nat. Electron.* **4**, 502–512 (2021).
13. Borders, W. A. et al. Integer factorization using stochastic magnetic tunnel junctions. *Nature* **573**, 390–393 (2019).
14. Aramon, M. et al. Physics-inspired optimization for quadratic unconstrained problems using a digital annealer. *Front. Phys.* **7**, 48 (2019).
15. Yamamoto, K. et al. 7.3 STATICA: a 512-spin 0.25M-weight full-digital annealing processor with a near-memory all-spin-updates-at-once architecture for combinatorial optimization with complete spin-spin interactions. In *2020 IEEE International Solid-State Circuits Conference—(ISSCC)* 138–140 (IEEE, 2020).
16. Patel, S., Canoza, P. & Salahuddin, S. Logically synthesized and hardware-accelerated restricted Boltzmann machines for combinatorial optimization and integer factorization. *Nat. Electron.* **5**, 92–101 (2022).

17. Su, Y., Mu, J., Kim, H. & Kim, B. A 252 spins scalable CMOS Ising chip featuring sparse and reconfigurable spin interconnects for combinatorial optimization problems. In *2021 IEEE Custom Integrated Circuits Conference (CICC)* 1–2 (IEEE, 2021).
18. Smithson, S. et al. Efficient CMOS invertible logic using stochastic computing. *IEEE Trans. Circuits Syst. I, Reg. Papers* **66**, 2263–2274 (2019).
19. Cai, F. et al. Power-efficient combinatorial optimization using intrinsic noise in memristor Hopfield neural networks. *Nat. Electron.* **3**, 409–418 (2020).
20. Tatsumura, K., Yamasaki, M. & Goto, H. Scaling out Ising machines using a multi-chip architecture for simulated bifurcation. *Nat. Electron.* **4**, 208–217 (2021).
21. Dixit, V., Selvarajan, R., Alam, M. A., Humble, T. S. & Kais, S. Training restricted Boltzmann machines with a D-Wave quantum annealer. *Front. Phys.* **9**, 589626 (2021).
22. Koller D. & Friedman, N. *Probabilistic Graphical Models: Principles and Techniques* (MIT Press, 2009).
23. Finocchio, G. et al. The promise of spintronics for unconventional computing. *J. Magn. Magn. Mater.* **521**, 167506 (2021).
24. Andriyash, E. et al. *Boosting Integer Factoring Performance via Quantum Annealing Offsets*. Report No. 14 (D-Wave Technical Report Series, 2016).
25. Dridi, R. & Alghassi, H. Prime factorization using quantum annealing and computational algebraic geometry. *Sci. Rep.* **7**, 43048 (2017).
26. Jiang, S., Britt, K. A., McCaskey, A. J., S Humble, T. & Kais, S. Quantum annealing for prime factorization. *Sci. Rep.* **8**, 17667 (2018).
27. Lucas, A. Ising formulations of many NP problems. *Front. Phys.* **2**, 5 (2014).
28. Camsari, K. Y. et al. Stochastic p-bits for invertible logic. *Phys. Rev. X* **7**, 031014 (2017).
29. Onizawa, N. et al. A design framework for invertible logic. In *2019 53rd Asilomar Conference on Signals, Systems, and Computers* 312–316 (IEEE, 2019).
30. Brélaz, D. New methods to color the vertices of a graph. *Commun. ACM* **22**, 251–256 (1979).
31. De Sa, C., Re, C. & Olukotun, K. Ensuring rapid mixing and low bias for asynchronous Gibbs sampling. In *Proc. 33rd International Conference on Machine Learning* 1567–1576 (PMLR, 2016).
32. Ko, G. G., Chai, Y., Rutenbar, R. A., Brooks, D. & Wei, G.-Y. FlexGibbs: reconfigurable parallel Gibbs sampling accelerator for structured graphs. In *2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)* 334 (IEEE, 2019).
33. Fang, Y. et al. Parallel tempering simulation of the three-dimensional Edwards–Anderson model with compact asynchronous multispin coding on GPU. *Comput. Phys. Commun.* **185**, 2467–2478 (2014).
34. Yang, K., Chen, Y.-F., Roumpos, G., Colby, C. & Anderson, J. High performance Monte Carlo simulation of Ising model on TPU clusters. In *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis* 83 (ACM, 2019).
35. Yoshimura, C., Hayashi, M., Okuyama, T. & Yamaoka, M. FPGA-based annealing processor for Ising model. In *2016 Fourth International Symposium on Computing and Networking (CANDAR)* 436–442 (IEEE, 2016).
36. Kaminsky, W. M. & Lloyd, S. Scalable architecture for adiabatic quantum computing of NP-hard problems. in *Quantum Computing and Quantum Bits in Mesoscopic Systems* 229–236 (Springer, 2004).
37. Block, B., Virnau, P. & Preis, T. Multi-GPU accelerated multi-spin Monte Carlo simulations of the 2D Ising model. *Comput. Phys. Commun.* **181**, 1549–1556 (2010).
38. Preis, T., Virnau, P., Paul, W. & Schneider, J. J. GPU accelerated Monte Carlo simulation of the 2D and 3D Ising model. *J. Comput. Phys.* **228**, 4468–4477 (2009).
39. Bhatti, S. et al. Spintronics based random access memory: a review. *Mater. Today* **20**, 530–548 (2017).
40. Safranski, C. et al. Demonstration of nanosecond operation in stochastic magnetic tunnel junctions. *Nano Lett.* **21**, 2040–2045 (2021).
41. Hayakawa, K. et al. Nanosecond random telegraph noise in in-plane magnetic tunnel junctions. *Phys. Rev. Lett.* **126**, 117202 (2021).
42. Sutton, B. et al. Autonomous probabilistic coprocessing with petaflips per second. *IEEE Access* **8**, 157238–157252 (2020).
43. Mohseni, M. et al. Nonequilibrium Monte Carlo for unfreezing variables in hard combinatorial optimization. Preprint at https://arxiv.org/abs/2111.13628 (2021).
44. Mosca, M., Marcos Vensi Basso, J. & Verschoor, S. R. On speeding up factoring with quantum SAT solvers. *Sci. Rep.* **10**, 15022 (2020).
45. Hoos, H. H. & Stützle, T. SATLIB: an online resource for research on SAT. *Sat* **2000**, 283–292 (2000).
46. Fleury, A. B. K. F. M. & Heisinger, M. CaDiCaL, KISSAT, PARACOOBA, PLINGELING and TREENGELING entering the SAT competition 2020. *SAT COMPETITION* **2020**, 50 (2020).
47. Soos, M., Devriendt, J., Gocht, S., Shaw, A. & Meel, K. S. CryptoMiniSat with CCAnr at the SAT competition 2020. *SAT COMPETITION* **2020**, 27 (2020).
48. Zhang, X., Bashizade, R., Wang, Y., Mukherjee, S. & Lebeck, A. R. Statistical robustness of Markov chain Monte Carlo accelerators. In *Proc. 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* 959–974 (ACM, 2021).
49. Biere, A. CaDiCaL, LINGELING, PLINGELING, TREENGELING and YALSAT entering the SAT competition 2017. In *Proc. SAT Competition* 13–14 (2017).

Acknowledgements

We are grateful to B. M. Sutton, D. Eppens, A. Ho and M. Mohseni for useful discussions. N.A.A. is grateful to G. Eschemann for useful discussions on airhdl. We acknowledge Xilinx for the hardware support. K.Y.C. and L.T. acknowledge support from the Institute of Energy Efficiency, University of California, Santa Barbara. K.Y.C. and N.A.A. acknowledge support from the National Science Foundation through CCF 2106260. The research of A.G., M.C. and G.F. has been supported by project no. PRIN 2020LWPKH7 funded by the Italian Ministry of University and Research and by Petaspin association (https://www.petaspin.com/). Computational facilities were purchased with funds from the National Science Foundation (CNS-1725797) and administered by the Center for Scientific Computing (CSC). The CSC is supported by the California NanoSystems Institute and the Materials Research Science and Engineering Center (MRSEC; NSF DMR 1720256) at University of California, Santa Barbara.

Author contributions

N.A.A. and K.Y.C. conceived the idea and planned the study. N.A.A. set up the FPGA and performed the experimental measurements. A.G., N.A.A., G.F. and K.Y.C. performed the data analysis. All the authors have participated in useful discussions and in writing the manuscript.

Competing interests

The authors declare no competing interests.

Additional information

Supplementary information The online version contains supplementary material available at https://doi.org/10.1038/s41928-022-00774-2.

Correspondence and requests for materials should be addressed to Navid Anjum Aadit, Giovanni Finocchio or Kerem Y. Camsari.

Peer review information *Nature Electronics* thanks Kosuke Tatsumura, Masanao Yamaoka and the other, anonymous, reviewer(s) for their contribution to the peer review of this work.